uncompressed size of the bits array, `m`, from which in turn depends the running time of look-ups through the formulas we developed in section 4.10, and on the other hand keep its compressed size small, from which depends the transfer ratio.

Since router tables will be updated periodically, decompressing the whole filter could be a heavy overhead for nodes: a good compromise is breaking the filter into pieces and compress each piece independently; this makes the overall compression rates slightly worse, but when updates are frequent (compared to lookups) it reduces even more the overhead for decompression, since each proxy, between two updates, won't decompress the whole bits array, but only the chunks it needs.

4.11.5 Scalable Bloom Filter

Yet another combination of several Bloom filters, working similarly as a layered Bloom filter. Only this time the different layers have increasing capacity (and hence smaller false positives ratio). This allows the container to adapt dynamically to the number of elements stored and keeps the probability of false positives low.

4.12 Summary

In this chapter, we have met a different kind of data-structure: all the ones we had described before were always returning the correct and best possible value, for instance a stack would always return the last item inserted, and a